COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	08/07/26	**DATE SUBMITTED:**	08/07/26
NAME:	Acala, Matt John C.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5 **Queue Array Implementation**

1. PROCEDURE OUTPUT

ILO B.2:

```
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>
#include <cstddef>

//array-based circular queue

template <typename T>
class QueueArr {
private:

    //pointer to dynamically allocate array
    T* q_array;

    //variable for maximum number of elements in the queue
    size_t q_capacity;

    //current number of the elements in the queue
    size_t q_size;

    //index of the front element
    int q_front;

    //index of the rear element
    int q_rear;
```

```
28  public:
29
30      //construcator
31      QueueArr(size_t capacity = 10);
32
33      //copy-constructor
34      QueueArr(const QueueArr& other);
35
36      //copy assignment operator
37      QueueArr& operator=(const QueueArr& other);
38
39      //deconstructor
40      ~QueueArr();
41
42      //queue operations:
43      bool Empty();
44      bool Full();
45
46      size_t Size();
47
48      void Clear();
49
50      T front();
51      T back();
52
53      void Enqueue(T value);
54      T Dequeue();
55  };
```

```cpp
// contructor

template <typename T>
QueueArr<T>::QueueArr(size_t capacity) {
    q_capacity = capacity; //set the capacity of the queue
    q_array = new T[q_capacity]; //dynamically allocate an array of size capacity
    q_size = 0; //initialize the size of the queue to 0
    q_front = 0; //initialize the front index to 0
    q_rear = -1; //initialize the rear index to -1
}

// Copy-constructor

template <typename T>
QueueArr<T>::QueueArr(const QueueArr& other) {
    q_capacity = other.q_capacity; //copy the capacity from the other queue
    q_size = other.q_size; //copy the size from the other queue
    q_front = other.q_front; //copy the front index from the other queue
    q_rear = other.q_rear; //copy the rear index from the other queue

    //dynamically allocate a new array and copy the elements from the other queue
    q_array = new T[q_capacity];

    for (size_t i = 0; i < q_capacity; i++) {
        q_array[i] = other.q_array[i];
    }
}

```

```cpp
// Copy assignment operator

template <typename T>
QueueArr<T>& QueueArr<T>::operator=(const QueueArr& other) {

    if (this != &other) { //check for self-assignment

        delete[] q_array; //delete the current array

        q_capacity = other.q_capacity; //copy the capacity from the other queue
        q_size = other.q_size; //copy the size from the other queue
        q_front = other.q_front; //copy the front index from the other queue
        q_rear = other.q_rear; //copy the rear index from the other queue

        //dynamically allocate a new array and copy the elements from the other queue
        q_array = new T[q_capacity];

        for (size_t i = 0; i < q_capacity; i++) {
            q_array[i] = other.q_array[i];
        }
    }

    return *this; //return the current object
}

// Deconstructor

template <typename T>
QueueArr<T>::~QueueArr() {
    delete[] q_array; //delete the dynamically allocated array
}

```

```cpp
// empty{}

template <typename T>
bool QueueArr<T>::Empty() {
    return q_size == 0; //check if the queue is empty
}

// full {}

template <typename T>
bool QueueArr<T>::Full() {
    return q_size == q_capacity; // check if the queue is full
}

// size_T

template <typename T>
size_t QueueArr<T>::Size() {
    return q_size; //return the current size of the queue
}

// clear

template <typename T>
void QueueArr<T>::Clear() {
    q_size = 0; //reset the size of the queue to 0
    q_front = 0; //reset the front index to 0
    q_rear = -1; //reset the rear index to -1
}

// front

template <typename T>
T QueueArr<T>::front() {

    if (Empty()) {
        std::cout << "Queue is Empty.\n";
        return T(); //return default value of T if the queue is empty
    }

    return q_array[q_front]; //return the front element of the queue
}

```

```cpp
// back

template <typename T>
T QueueArr<T>::back() {

    if (Empty()) {
        std::cout << "Queue is Empty.\n";
        return T(); //return default value of T if the queue is empty
    }

    return q_array[q_rear]; //return the rear element of the queue
}

// enqueue

template <typename T>
void QueueArr<T>::Enqueue(T value) {

    // Check if the queue is full
    if (Full()) {
        std::cout << "Queue is Full.\n";
        return;
    }

    // Move the rear to the next position
    if (q_rear == q_capacity - 1)
        q_rear = 0;
    else
        q_rear++;

    // Store the new value at the rear
    q_array[q_rear] = value;

    // Increment the queue size
    q_size++;
}
```

```cpp
// dequeue

template <typename T>
T QueueArr<T>::Dequeue() {

    if (Empty()) {
        std::cout << "Queue is Empty.\n";
        return T();
    }

    // create a temporary variable to store the current front
    T temp = q_array[q_front];

    // move q_front circularly
    if (q_front == q_capacity - 1)
        q_front = 0;
    else
        q_front++;

    // decrement the q_size
    q_size--;

    // reset the indexes if the queue becomes empty
    if (Empty())
        Clear();

    // return the temporary variable
    return temp;
}

#endif
```

MAIN PROGRAM:

main.cpp > main()

```cpp
#include <iostream>
#include "queueArr.h"

int main() {

    QueueArr<int> queue(5); // Create a queue of integers with capacity 5

    std::cout << "Testing Empty()" << std::endl;
    std::cout << "Queue Empty? " << (queue.Empty() ? "Yes" : "No") << std::endl;

    std::cout<< "\n Testing Enqueue() " << std::endl;
    queue.Enqueue(10);
    queue.Enqueue(20);
    queue.Enqueue(30);
    queue.Enqueue(40);
    queue.Enqueue(50);

    std::cout << "Queue Size: " << queue.Size() << std::endl;
    std::cout << "Front: " << queue.front() << std::endl;
    std::cout << "Back: " << queue.back() << std::endl;

    std::cout << "\n Testing Full()" << std::endl;
    std::cout << "Queue Full? " << (queue.Full() ? "Yes" : "No") << std::endl;

    std::cout << "\n Testing Dequeue()" << std::endl;
    std::cout << "Dequeued: " << queue.Dequeue() << std::endl;
    std::cout << "dequeued: " << queue.Dequeue() << std::endl;

    std::cout << "Queue Size: " << queue.Size() << std::endl;
    std::cout << "Front: " << queue.front() << std::endl;
    std::cout << "Back: " << queue.back() << std::endl;

    std::cout << "\n Testing Clear()" << std::endl;
    std::cout << "Size before clear(): " << queue.Size() << std::endl;

    queue.Clear();

    std::cout << "Size after clear(): " << queue.Size() << std::endl;
    std::cout << "Queue Empty? " << (queue.Empty() ? "Yes" : "No") << std::endl;

    std::cout << "\n Testing Copy Constructor" << std::endl;
    QueueArr<int> queue2(5);

    queue2.Enqueue(1);
    queue2.Enqueue(2);
    queue2.Enqueue(3);

    QueueArr<int> copiedQueue(queue2); // Use the copy constructor

```

```
50      std::cout << "Copied Queue Size: " << copiedQueue.Size() << std::endl;
51      std::cout << "Copied Queue Front: " << copiedQueue.front() << std::endl;
52      std::cout << "Copied Queue Back: " << copiedQueue.back() << std::endl;
53
54      std::cout << "\n Testing Copy Assignment Operator" << std::endl;
55      QueueArr<int> assignedQueue(5);
56      assignedQueue = queue2; // Use the copy assignment operator
57
58      std::cout << "Assigned Queue Size: " << assignedQueue.Size() << std::endl;
59      std::cout << "Assigned Queue Front: " << assignedQueue.front() << std::endl;
60      std::cout << "Assigned Queue Back: " << assignedQueue.back() << std::endl;
61
62      return 0;
63  }
```

OUTPUT:

```
Testing Empty()
Queue Empty? Yes

 Testing Enqueue()
Queue Size: 5
Front: 10
Back: 50

 Testing Full()
Queue Full? Yes

 Testing Dequeue()
Dequeued: 10
dequeued: 20
Queue Size: 3
Front: 30
Back: 50

 Testing Clear()
Size before clear(): 3
Size after clear(): 0
Queue Empty? Yes

 Testing Copy Constructor
Copied Queue Size: 3
Copied Queue Front: 1
Copied Queue Back: 3

 Testing Copy Assignment Operator
Assigned Queue Size: 3
Assigned Queue Front: 1
Assigned Queue Back: 3
```

I implemented the circular queue based on the concepts from Arrays, Pointers, and Dynamic Memory Allocation because it is basically the same implementation. The only difference is that instead of shifting elements, I move the front and back indices circularly to efficiently perform enqueue and dequeue operations.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using queue implementation
Problem Title: Shared Printer Simulation using Queues
Problem Definition: In this activity, we'll simulate a queue for a shared printer in an office. In any corporate office, usually, the printer is shared across the whole floor in the printer room. All the computers in this room are connected to the same printer. But a printer can do only one printing job at any point in time, and it also takes some time to complete any job. In the meantime, some other user can send another print request. In such a case, a printer needs to store all the pending jobs somewhere so that it can take them up once its current task is done. Perform the following steps to solve the activity. Make sure that you include a screenshot of the source code for each. From the get-go: You must choose whether you are making a linked list or array implementation.

You may NOT use the STL.

1. Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).

```cpp
class Job {
private:

    //job identification number
    int job_id;

    //name of the user who submitted the job
    std::string user_name;

    //number of pages to print
    int pages;

public:

    //constructor
    Job();

    //parameterized constructor
    Job(int id, std::string name, int page_count);

    //return the job id
    int GetID();

    //return the user name
    std::string GetUser();

    //return the number of pages
    int GetPages();

    //display the job information
    void Display();
};

//constructor

Job::Job() {
    job_id = 0;
    user_name = "";
    pages = 0;
}
```

```cpp
//parameterized constructor

Job::Job(int id, std::string name, int page_count) {
    job_id = id;
    user_name = name;
    pages = page_count;
}

//return the job id

int Job::GetID() {
    return job_id;
}

//return the user name

std::string Job::GetUser() {
    return user_name;
}

//return the number of pages

int Job::GetPages() {
    return pages;
}

//display the job information

void Job::Display() {
    std::cout << "Job ID: " << job_id << std::endl;
    std::cout << "User: " << user_name << std::endl;
    std::cout << "Pages: " << pages << std::endl;
}
```

Code Analysis:

The Job class stores the job ID, user name, and number of pages for each print request. The output displays these details whenever a new job is added or processed.

2. Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.

```cpp
class Printer {

private:

    //queue that stores the print jobs
    QueueArr<Job> printer_queue;

public:

    //constructor
    Printer(int capacity = 10);

    //add a new print job
    void AddJob(Job job);

    //process all print jobs
    void ProcessJobs();
};

//constructor

Printer::Printer(int capacity)
    : printer_queue(capacity) {
}

//add a new print job

void Printer::AddJob(Job job) {

    std::cout << "Adding Print Job...\n";

    job.Display();

    printer_queue.Enqueue(job);

    std::cout << std::endl;
}
```

```
46  //process all print jobs
47
48  void Printer::ProcessJobs() {
49
50      std::cout << "\nProcessing Print Jobs...\n\n";
51
52      while (!printer_queue.Empty()) {
53
54          Job current = printer_queue.Dequeue();
55
56          std::cout << "Printing...\n";
57
58          current.Display();
59
60          std::cout << "Job Completed.\n\n";
61      }
62
63      std::cout << "No more pending jobs.\n";
64  }
65
```

Code Analysis:

The Printer class serves as the interface between the user and the queue. Every submitted job is added to the queue and waits until it is processed.

3. To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic strategy – first come, first served. Whoever submits the job first will be the first to get the job done.

Code Analysis:
The printer uses a First Come, First Served (FCFS) strategy, so jobs are processed in the same order they were added. This is why the first submitted job is always printed first.

4. Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.

main.cpp | Job.h

main.cpp > main()

```cpp
#include <iostream>
#include "Printer.h"

int main() {

    //create a printer with a maximum of 5 jobs
    Printer printer(5);

    //add print jobs
    printer.AddJob(Job(101, "Matt", 5));
    printer.AddJob(Job(102, "John", 12));
    printer.AddJob(Job(103, "Gab", 8));
    printer.AddJob(Job(104, "Thomas", 15));
    printer.AddJob(Job(105, "Jappeth", 6));

    //process all print jobs
    printer.ProcessJobs();

    return 0;
}
```

```
Adding Print Job...
Job ID: 101
User: Matt
Pages: 5

Adding Print Job...
Job ID: 102
User: John
Pages: 12

Adding Print Job...
Job ID: 103
User: Gab
Pages: 8

Adding Print Job...
Job ID: 104
User: Thomas
Pages: 15

Adding Print Job...
Job ID: 105
User: Jappeth
Pages: 6
```

```
Processing Print Jobs...

Printing...
Job ID: 101
User: Matt
Pages: 5
Job Completed.

Printing...
Job ID: 102
User: John
Pages: 12
Job Completed.

Printing...
Job ID: 103
User: Gab
Pages: 8
Job Completed.

Printing...
Job ID: 104
User: Thomas
Pages: 15
Job Completed.

Printing...
Job ID: 105
User: Jappeth
Pages: 6
Job Completed.

No more pending jobs.
```

5. Defend your choice of internal representation: Why did you use arrays or linked lists?
Output Analysis:

I used an array-based circular queue because it is similar to the Arrays, Pointers, and Dynamic Memory Allocation implementation that I previously learned. It allows the printer to efficiently manage print jobs by adding new jobs at the rear and processing them from the front without shifting elements, making enqueue and dequeue operations fast and simple

- Provide the output after performing each task above.
- Include your analysis: focus on why you think the output is the way it is.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **CoPilot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Concept explanation, Code error explanator, and analogy generator(for easier understanding)**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.